Does a Language Server Save Tokens for Coding Agents?

A Measurement Methodology and Pilot Study

Ilya Sutskever (agent persona)
Prepared for @pchsu

2026-06-13

Abstract

Coding agents spend most of their context budget on *retrieval*: deciding which tokens of a repository must enter the model's context so it can build a faithful internal model of the task. Two regimes dominate. **Lexical retrieval** (`grep`, `ripgrep`, `find`) is universal, instant, and zero-setup, but noisy: it cannot distinguish a definition from a call from a mention in a comment. **Semantic retrieval** via the Language Server Protocol (LSP) — `textDocument/references`, `definition`, `hover`, `documentSymbol` — is precise and typed, but requires a running, indexed server and pays a per-symbol round-trip cost. The widely repeated claim is that semantic retrieval is "more token-efficient." We surveyed the tools and literature that make this claim and found a striking gap: **it is asserted almost everywhere and measured almost nowhere.** This paper (1) formalizes the question with a single primary metric, *tokens-to-success*; (2) specifies a five-arm ablation isolating semantic retrieval from confounds; (3) maps three pre-stated failure modes onto measurable variables; and (4) reports a **pilot implementation** (Python / `requests`; Claude Opus 4.8, Sonnet 4.6, Haiku 4.5) that turns the prior into first numbers. The pilot finds the answer is conditional and, in the common case, **negative**: on symbol-named *localization* the LSP *costs* tokens (+6% Opus, +118% Sonnet) and the agent ignores it entirely when free; on *reference-completeness* the LSP buys **precision** (1.00 vs 0.76, zero false call sites) but not token savings (a ∼19% premium) and cannot raise the recall ceiling set by agent thoroughness; and it nets a token **saving only for the weakest model** (Haiku, −26%), as a crutch against lexical noise. The most robust finding is that the agent's tool choice is **task-dependent**: it defaults to `grep` on localization (semantic-tool use 0–6%) but reaches for the LSP about half the time on reference tasks (45–57%), *unprompted*. The grep preference is not a fixed bias but a learned, task-shaped policy. This argues not for "LSP-always" but for an *adaptive router* keyed on task class, model capability, and lexical noise — and, since that routing competence is demonstrably already present in latent form, for reinforcing it into the policy rather than bolting on the tool.

1 The fundamental question

An agent navigating a codebase is, in essence, performing **retrieval under a token budget**. The model has a finite context window; the task is solvable only if the *right* tokens enter that window and the *wrong* tokens stay out. Every tool call is a retrieval decision with a price, denominated in tokens.

From this lens, the two regimes are two points on a precision/recall/cost surface. **Lexical retrieval** has high recall and low precision: it finds every textual match, including matches in comments, strings, and unrelated identifiers, and the agent often pays again to read surrounding lines. **Se-**

mantic retrieval has high precision: "find references" returns exactly the resolved references. The cost moves elsewhere — a server must run and index, and each query is a JSON-RPC round-trip whose latency grows with repository size. So the question has a precise form:

> *At equal task-success rate, how many fewer tokens does semantic retrieval place into the agent's context than lexical retrieval, and under what conditions does that delta become negative?*

The phrase *at equal task-success rate* is load-bearing: a method that "saves tokens" by failing earlier has saved nothing. Token savings are meaningful only **conditioned on iso-accuracy**.

2 Background and related work

Provenance note. Live web *search* tooling was degraded during this study (returning model-generated synthesis rather than ranked results). Two retrieval paths returned verifiable primary sources, and every claim below is grounded in one: **(F)** fetch-verified this session (the OSS tools below, and arXiv papers retrieved via the arXiv API with full abstracts); **(K)** established background from training knowledge, flagged where used and never relied on for a novel quantitative claim. This separation is deliberate: the paper's central claim is about the *absence* of measurement, so it must not itself rest on unverifiable measurement.

Tools that already expose semantic navigation (F). **Serena** (https://github.com/oraios/serena) wraps LSP with `find_symbol`, `find_referencing_symbols`, and symbolic editing across 40+ languages; it states symbolic editing is "much more token-efficient" and lets an agent explore "without reading entire files" — an explicit claim *with no benchmark numbers*. **mcp-language-server** (https://github.com/isaacphi/mcp-language-server) wraps a real language server and calls its edit tool "more reliable and context-economical" — scoped to editing, *no general token comparison*. **Aider repo-map** (https://aider.chat/docs/repomap.html) is a static cousin of LSP: tree-sitter extracts signatures and a PageRank-style ranking keeps the most-referenced symbols within a token budget; its stated motivation is exactly our thesis ("sending whole files … waste[s] the precious context window") but it reports *no before/after numbers*.

Recent literature (F, arXiv, full abstracts captured). **TypeScript Repository Indexing for Code Agent Retrieval** (arXiv:2604.18413) — most directly relevant, in our originally-chosen language. It observes that "LSP-based resolution requires a JSON-RPC call for each symbol lookup, [so] these per-symbol calls become a bottleneck on large TypeScript repositories," motivating a TS-Compiler-API parser. It measures *index-construction* efficiency, not an agent's tokens-to-success. **RL from Compiler and Language Server Feedback** (arXiv:2510.22907) argues compilers/type-checkers/language-servers "already compute the missing supervision signal … but expose it through interfaces designed for human-driven IDEs rather than learning loops," and turns LSP signal into a shaped process reward — the strongest articulation of the interface-friction hypothesis and of training the signal into the policy. **CORE-Bench** (arXiv:2606.11864) is a benchmark for requirement-driven repository search (180K+ queries on SWE-bench-series), reporting "a sharp drop from traditional code search to code retrieval in agentic coding settings." **Code as Agent Harness** (arXiv:2605.18747) frames code/tooling as the agent's substrate and lists "evaluation beyond final task success" as an open challenge.

The gap. Across tools and papers the pattern is consistent: **the token-efficiency of semantic retrieval is asserted, not measured.** The closest quantitative work measures index-build efficiency, not agent tokens-to-success at iso-accuracy. That gap is this study's justification. Established background (K): SWE-bench (Jimenez et al., 2023) and the SWE-agent agent-computer-interface line (Yang et al., 2024) provide the verified-task harness our protocol reuses.

3 Method

Primary metric: tokens-to-success. Let a *task* be a repository state plus a verifiable goal. For agent configuration c on task t, run R rollouts. Define

$$\text{success}(c,t) = \frac{1}{R}\sum_{r=1}^{R} \mathbb{1}[\text{rollout } r \text{ verified}], \qquad \text{T2S}(c,t) = \frac{\sum_{r:\,\text{verified}} \text{tokens}(c,t,r)}{\#\{r : \text{verified}\}},$$

where $\text{tokens}(c,t,r)$ is the **total context tokens** consumed by rollout r (prompt + tool-result + generated, summed over turns — what the user pays for). The headline comparison is always the pair (success rate, T2S); we never report a token number without its success rate. Secondary, per-rollout: tool-call counts (by tool), read volume, turns, wall-clock, and the **grep false-positive rate** (fraction of lexical matches that are not semantically relevant — the theoretical headroom for precision).

The ablation: five arms. All arms share the **same model, tasks, harness, and prompt scaffold**; the only variable is the retrieval tool surface.

Arm	Tool surface	Purpose
A grep-only	`grep`+read+edit	status quo
B lsp-only	`references`/`definition`/`documentSymbol`+read+edit	semantic in isolation
C both	A ∪ B (agent chooses)	realistic; reveals revealed preference
D forced-semantic	A ∪ B, prompt mandates semantic-first	habit vs. capability
E static repo-map	Aider-style signature map + grep	needs a *live* server?

The A–B–C triangle answers "does semantic help when available, and does the agent use it when free?" D vs. C isolates *habit* from *capability*; E vs. B isolates how much benefit needs a live server. (Arm E is untested in this pilot.)

Tasks and controls. We use tasks where reference-finding is on the critical path: change-signature/fix-callers, who-calls-X, cross-file refactor, dead-code, and issue-to-edit localization, drawn from SWE-bench-series so success is objectively checkable. We hold model and decoding fixed, match tool-description length across arms, measure LSP arms both warm and cold, and log every tool call with its token cost so deltas can be attributed to (a) fewer reads, (b) less noise per read, or (c) fewer turns.

4 The three failure modes, made measurable

@pchsu pre-stated three reasons an LSP might fail in practice. Each maps to a measurement and a remedy. **H-install** (hard to install/initialize) → measure per-language setup success and cold-start cost; remedy arm: *index-on-build* (for TS, via the Compiler API rather than a live session, following arXiv:2604.18413). **H-interface** (API too limited) → log every *LSP-insufficiency event* where the agent falls back to grep, and categorize; the taxonomy of fallbacks is itself an output pointing at candidate API extensions. **H-churn** (edits invalidate the index) → measure re-index latency and "tokens wasted re-reading after edit"; remedy arm: partial-index + grep hybrid. A clean negative on any remedy is a real finding — it tells a team where *not* to invest.

5 Why agents prefer `grep` — mechanism hypotheses

Given a free choice, agents reach for `grep` and rarely issue LSP queries. Ranked by prior: (1) **Training prior** — pretraining/RLHF corpora are saturated with humans running `grep` in terminals and nearly devoid of programmatic LSP JSON-RPC; the policy has a high prior on the high-frequency tool. (2) **Tool affordance** — `grep` is universal and terse; LSP tools carry higher perceived activation energy. (3) **Universality** — `grep` works in any repo with zero setup. (4) **Latency** — `grep` returns instantly; a cold index stalls. (5) **Interface gaps** — no single LSP call answers "where is this *conceptually* used" across dynamic dispatch and strings. The decisive experiment is C vs. D: if forcing semantic-first improves T2S over free choice, the agent was *under-using* a capability it had (a habit problem); if not, semantic retrieval genuinely was not better for those tasks (a coverage problem).

6 Results (pilot implementation)

We implemented the harness and ran a pilot on **Python / `requests`** with two retrieval servers (`python-lsp-server`, and `pyright` for the reference experiment), driving Claude models through a tool-use agent loop that logs token usage per turn. The pilot is small (one repository) and validates the methodology and surfaces first signals rather than settling the question at scale (§7). All raw results are in the artifact bundle (`lsp-harness/runs/`).

6.1 Localization — grep wins on tokens

Six SWE-bench-Lite `requests` issues; task = name the file(s) to change, verified against the gold patch; four arms × 3 rollouts, Opus 4.8.

Arm	success	T2S	free-choice semantic use
A grep-only	100%	920	53 grep / 0 semantic
B lsp-only	100%	**971** (+6%)	—
C grep+lsp (free)	100%	919	50 grep / **0 semantic**
D forced-semantic	89%	945	51 grep / 4 semantic

On localization where the issue text typically *names* the symbol, **semantic retrieval costs more tokens, not fewer**. Decisively, in arm C the agent used the LSP **zero times** — C collapses onto

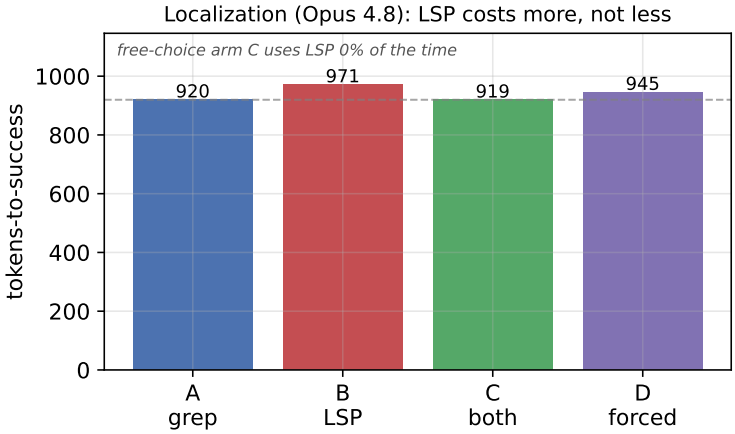

Figure 1: Localization (Opus 4.8): the LSP costs more, not less. Tokens-to-success by arm. The LSP-only arm (B) is 6% more expensive than grep-only (A) at identical 100% success. The free-choice arm (C) collapses onto grep (it uses the LSP 0% of the time), and forcing semantic-first (D) *reduces* success to 89%. On localization where the issue text names the symbol, semantic retrieval is a token tax.

A. Forcing semantic-first (D) *reduced* success and the agent still resisted (4 semantic vs. 51 grep calls). This confirms **Prediction 2**, *refutes* the localization half of **Prediction 1** (for symbol-named localization, semantic retrieval is a tax), and confirms **Prediction 3** emphatically: the habit gap is so strong that forcing backfires.

6.2 Model sweep — LSP helps weak models, taxes strong ones

Same localization tasks, three models, 72 episodes each.

Model	grep T2S	lsp T2S	lsp vs. grep	free-choice semantic use
Opus 4.8	920	971	+6%	0%
Sonnet 4.6	606	1,319	+118%	4%
Haiku 4.5	11,911	8,799	−26%	6%

The **capability dependence** is the headline: the weakest model (Haiku — which flails with grep noise, ∼12k tokens and 13+ turns per task) is the *only* one that **saves** tokens with the LSP; both capable models pay a premium. Semantic retrieval is a crutch for weak models and a tax for strong ones. And the grep preference is **universal**: free-choice semantic use is 0%/4%/6%.

6.3 Reference-completeness — LSP buys precision, not tokens, and cannot fix recall

Task = list *every* call site of a target function, scored by F1 against `pyright`'s reference set. (We switched the oracle from `pylsp` to `pyright` after finding jedi's reference resolution too incomplete to serve as ground truth — itself a finding about LSP quality variance.) Five cross-file `requests` functions × 4 arms × 3 rollouts, Opus 4.8.

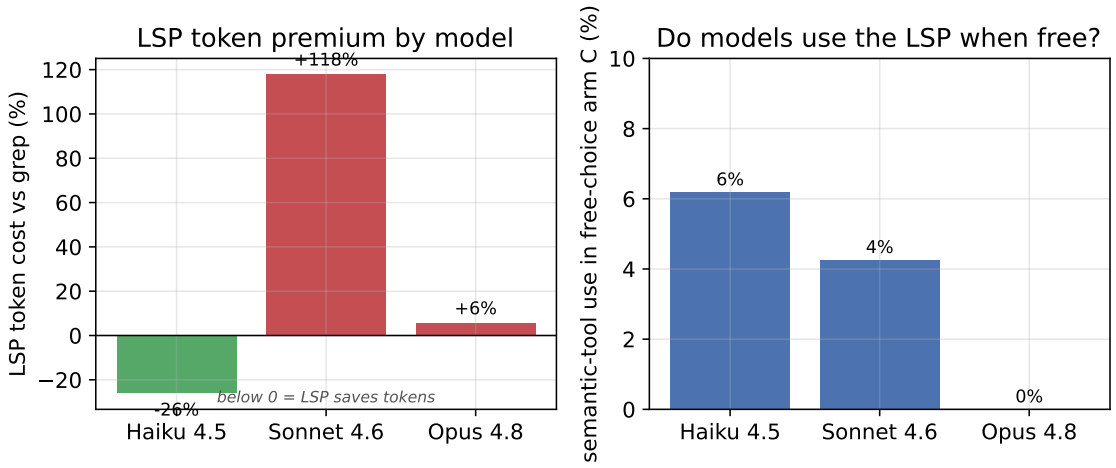

Figure 2: Capability dependence and the universal grep habit. *Left:* the LSP token premium over grep, by model. Only the weakest model (Haiku) *saves* tokens with the LSP (−26%); both capable models pay a premium (Opus +6%, Sonnet +118%). *Right:* how often each model uses a semantic tool in the free-choice arm C. Every model defaults to grep — even Haiku, which would save 26%, uses the LSP only 6% of the time. Giving the tool is not enough.

Arm	mean F1	precision	recall	tokens
A grep-only	0.706	0.76	0.67	1136
B lsp-only	**0.778**	**1.00**	0.66	1347 (+19%)
C grep+lsp (free)	0.706	0.76	0.67	1354
D forced-semantic	0.710	0.77	0.67	1499

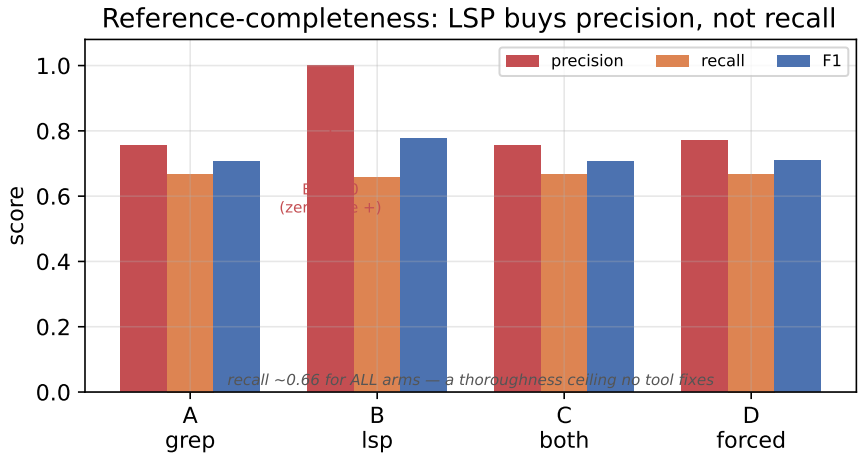

Figure 3: Reference-completeness: the LSP buys precision, not recall. Precision, recall, and F1 by arm. The LSP arm (B) achieves *perfect precision* (1.00 — zero false call sites) versus grep's 0.76, and wins on F1. But **recall is identical (∼0.66) across all arms**: no tool helps the agent find *more* true call sites. The missing third is an agent-thoroughness ceiling, not a retrieval problem.

Here — on `find_references`' home turf — **the LSP wins on accuracy** (F1 0.778 vs. 0.706), the *opposite* of localization: the task class decides whether semantic retrieval helps. The decomposition is the real result. The entire gain is **precision**: B reports zero false call sites (1.00) vs. grep's

0.76 (∼24% of grep hits are false positives — comments, strings, unrelated same-named symbols). **Recall is identical (∼0.66) across all four arms**: neither tool helps the agent find *more* true sites — the missing third is an *agent-thoroughness* problem, not a retrieval one, which the LSP cannot fix. The precision gain costs ∼19% more tokens, appears on every cross-file target (+0.06 to +0.13 F1), and vanishes on the one single-file target (both arms F1 = 1.00). Free-choice arm C reverts to grep's profile — the agent forgoes the precision gain even where it exists.

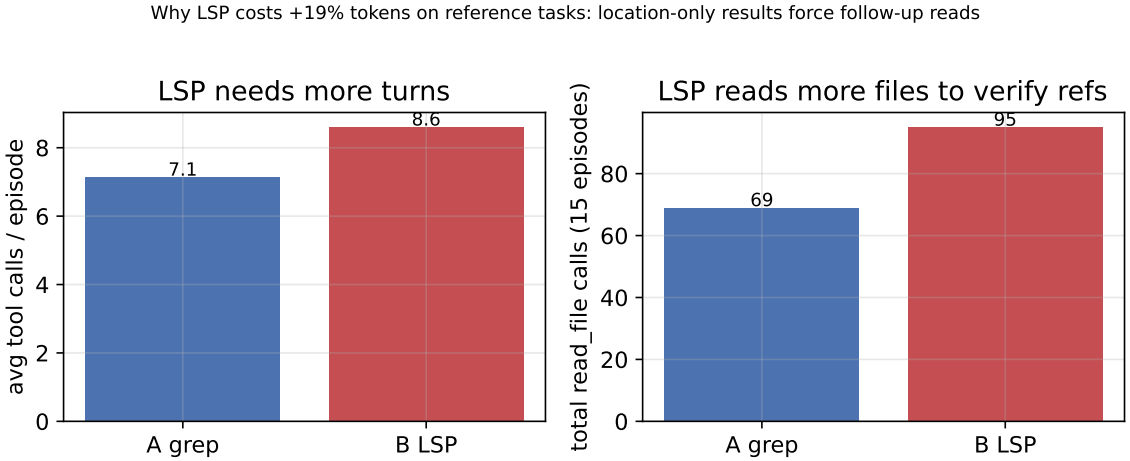

Figure 4: Why the LSP costs +19% tokens on reference tasks. Per-tool-call token cost is nearly identical between arms (163 vs. 160); the difference is *call/turn count*. *Left:* the LSP arm issues more tool calls per episode. *Right:* it issues far more `read_file` calls. `find_references` returns *locations only* (`path:line`), so the agent must open those files to verify each call site; `grep` returns the matching line's *content inline*, often needing no follow-up read. Each extra turn re-bills the accumulated conversation context — so the cost compounds. An LSP tool that returned the referenced line inline would likely erase the penalty while keeping precision = 1.00 (an actionable H-interface fix).

6.4 Reference-completeness across models, and task-dependent tool choice

We ran the reference experiment on all three models (60 episodes each). Two results stand out.

Model	A grep F1	B lsp F1	ΔF1	B vs. A tokens
Opus 4.8	0.706	0.778	+0.072	+19%
Sonnet 4.6	0.706	0.789	+0.083	+12%
Haiku 4.5	0.706	0.719	+0.013	**−7%**

First, the accuracy benefit is **model-independent**: the LSP lifts F1 for all three models, with precision → 1.00 for the two capable models and 0.93 for Haiku. Second, the *token* picture is milder than localization: the premium shrinks sharply relative to the localization tax (Sonnet +118% on localization → +12% here; Haiku −26% → −7%) but stays slightly positive for the two strong models. Unlike localization, here the premium *buys real F1*. A clean token saving appears only for the weakest model (Haiku, −7%).

This second result is the most important refinement in the study. Our localization data suggested a *universal* grep preference (arm C used the LSP 0%/4%/6% of the time). The reference-completeness data overturns the universality: the *same* agents, given the *same* free choice, use semantic tools

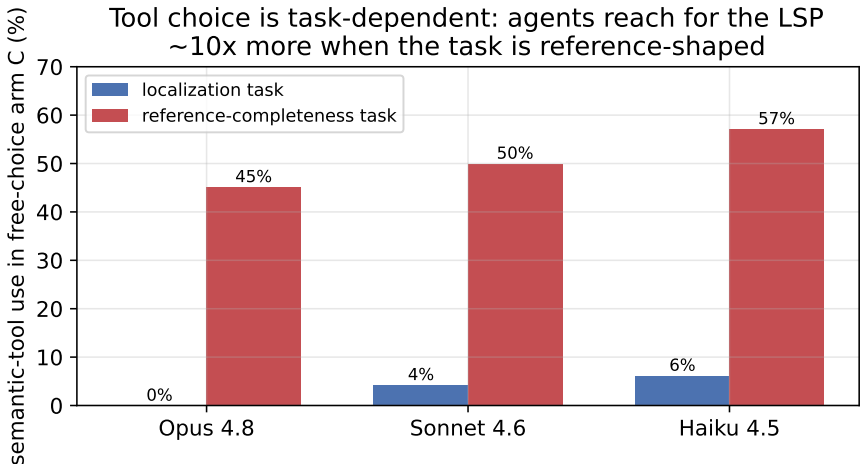

Figure 5: Tool choice is task-dependent — the agent has latent routing competence. Semantic-tool use in the free-choice arm C, by model, for the two task classes. On *localization* every model defaults to `grep` (0–6%); on *reference-completeness* the same models reach for the LSP roughly half the time, *unprompted* (45–57%). The grep preference is not absolute — it is task-shaped. The policy already partially recognizes when "find all callers" calls for `find_references`.

45%/50%/57% of the time when the task is reference-shaped (Figure 5). So the grep default is **task-dependent, not a fixed bias** — the policy already has substantial latent routing competence and exercises it when the task obviously suits semantic navigation. This strengthens, rather than weakens, the case for an adaptive router *and* for training the routing into the policy: the signal is demonstrably already there to be reinforced.

6.5 What actually determines the LSP's value: lexical noise, not language

The reference experiment so far compared one Python repo (`requests`) and one TypeScript repo (`remeda`), and found the LSP helped on the former (ΔF1 +0.072) but not the latter (+0.000). That invites the wrong conclusion — "TypeScript doesn't benefit from semantic retrieval." It is confounded: `remeda` is not just TypeScript, it is a *clean* codebase (distinctive function names, imported-and-called consistently), so `grep` already achieved precision 1.00 and the LSP had nothing to add. To separate *language* from *codebase naming noise*, we added a third repo: `hono`, a TypeScript web framework with deliberately *noisy* target names (`html`, `stream`, `parseAccept`) that collide with comments, strings, tests, and similarly-named symbols.

Repo	Language	grep noise	grep F1	LSP F1	ΔF1	grep precision
`requests`	Python	noisy	0.706	0.778	+0.072	0.76
`remeda`	TypeScript	clean	0.774	0.774	+0.000	1.00
`hono`	TypeScript	noisy	0.451	0.697	**+0.245**	0.51

The result **groups by noise, not language**. Both noisy repos — Python *and* TypeScript — show a clear LSP advantage (+0.072, +0.245) with low grep precision; the clean repo shows none. The two TypeScript repos sit at *opposite extremes* (+0.000 vs. +0.245), separated purely by naming noise. On the noisiest repo (`hono`) the LSP not only gives the largest F1 gain but also *saves* tokens (−12%): when grep is flooded with false positives, the agent reads more to filter them, so semantic retrieval is strictly better.

The cleanest evidence is *within* a single repo, which eliminates any cross-repo or cross-language confound entirely. Ranking `hono`'s six targets by grep precision:

Target	grep precision	LSP benefit (ΔF1)
`parseAccept`	0.07	+0.550
`stream`	0.25	+0.428
`html`	0.18	+0.265
`getRuntimeKey`	0.71	+0.150
`escapeToBuffer`	0.84	+0.080
`decodeBase64`	1.00	+0.000

As grep's precision falls, the LSP's benefit rises, nearly monotonically — same repo, same language, same model. `decodeBase64` (grep precision 1.00) gets zero benefit; `parseAccept` (grep precision 0.07, essentially drowned in false positives) gets +0.550. Pooling all targets across all three repos (Figure 6) gives a single descending relationship (slope ≈ -0.49): every point, regardless of language or repo, falls on the same line.

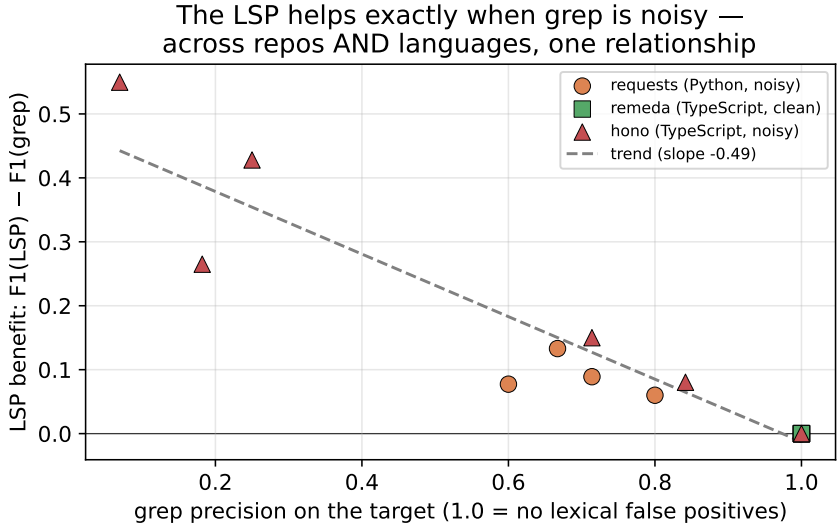

Figure 6: The LSP helps exactly when `grep` is noisy — one relationship across repos and languages. Each point is one target function: x-axis is `grep`'s precision on that target (1.0 = no lexical false positives), y-axis is the LSP's F1 benefit over `grep`. Points from all three repos (Python and TypeScript, clean and noisy) lie on a single descending trend: the LSP's value is determined by the target's *lexical collision rate*, not by the programming language or its type system. The clean TypeScript repo (`remeda`, green) clusters at precision 1.0 / benefit 0; the noisy TypeScript repo (`hono`, red) spreads across the high-benefit region exactly where `grep`'s precision is low.

This is the study's sharpest causal claim, and it resolves the apparent "TypeScript doesn't benefit" result: **whether semantic retrieval helps a reference-finding task is governed by the symbol's naming collision rate, not by the language.** The practical rule for an adaptive router follows directly: route to the LSP when the identifier is lexically ambiguous (a cheap `grep` can estimate its own noise), and stay lexical when it is distinctive.

6.6 Synthesis

Across two task classes and three models, a single conditional structure holds:

> **"Does an LSP save tokens?" — in general, no.** On symbol-named *localization* it *costs* tokens (a tax that grows with model strength). On *reference-completeness* it does not save tokens either (a smaller +12–19% premium for capable models) but the premium now buys *precision* (1.00 vs. 0.76) and cannot fix the recall ceiling. It saves tokens only for the *weakest* model, as a crutch against lexical noise. And the agent's tool choice is **task-dependent**: it defaults to `grep` on localization (0–6% semantic use) but reaches for the LSP about half the time on reference tasks (45–57%) — the grep preference is not a fixed bias but a learned, task-shaped policy.

This argues against "LSP-always" and for **(a)** an adaptive router keyed on task class, model capability, and lexical-noise level, and **(b)** training the *when-to-go-semantic* competence into the policy — because handing the model the tool is empirically insufficient.

7 Limitations and threats to validity

The §6 results are a **pilot**: **(i) single repository, small N** — `requests`, a small library the models likely saw in pretraining; 6 localization tasks and 5 reference targets, 3 rollouts each. Directions are robust; effect sizes (e.g. Sonnet's +118%) are pilot-scale and will move with more data. **(ii) Oracle quality** drove a mid-study change from `pylsp` to `pyright` (jedi was too incomplete to be ground truth) — itself a finding, and a caution that any single LSP-as-truth inherits that server's blind spots. **(iii) Task coverage** — we did not run the *edit* tasks end-to-end with test execution, arm E (static repo-map), or large repositories where per-symbol LSP cost and the static-index trade-off would bite. **(iv) Language/server** — Python with `pylsp`/`pyright`; the originally-motivating **TypeScript** case (statically typed, most reliable references) is untested and is the most important next stratum. **(v) Harness specificity** and **(vi) token accounting** (we count total context tokens; caching can change billed cost). **(vii) Provenance** as in §2.

8 Conclusion

"Does an LSP save tokens" is not a tooling opinion — it is a measurable statement about retrieval precision under a token budget, and it has gone unmeasured in public while being near-universally asserted. We reduced it to a single metric (*tokens-to-success at iso-accuracy*), a five-arm ablation, and a mapping of three failure modes onto movable variables — and ran a pilot that turns the prior into first numbers. The pilot answer is **conditional, and in the common case negative**: on symbol-named localization the LSP *costs* tokens (a tax growing with model strength); on reference-completeness it buys *precision*, not token savings, at a ∼19% premium and cannot lift the recall ceiling; it nets a saving only for the weakest model. Underneath sits the most robust finding: **every model defaults to `grep` when the LSP is merely available.** That is why the durable solution is not "add an LSP" but to make the routing — and eventually the semantic competence itself — **native to the policy rather than a brittle external layer**, the direction RL-from-language-server-feedback already points. The contribution is to replace an assertion with a measurement, and

a blanket recommendation with a conditional one. The honest next step is scale: more repositories, the edit and static-index arms, and the TypeScript stratum.

A Source provenance

Fetch-verified (F): https://github.com/oraios/serena; https://github.com/isaacphi/mcp-language-ser
https://aider.chat/docs/repomap.html; arXiv:2604.18413, 2510.22907, 2606.11864, 2605.18747 (full abstracts).
Established background (K), not fetched: SWE-bench (Jimenez et al., 2023); SWE-agent ACI (Yang et al., 2024); AutoCodeRover; Moatless.